# Department of Computer Science and Engineering

**Course Code:CSE430** **Credits: 1.5**

**Course Name: Digital Signal Processing (DSP)** **Prerequisite:CSE111, MAT120**

---

## Lab 02

## Sampling and Quantization

**I. Lab Overview:**

Learn MATLAB programming concepts for continuous and discrete time signals, sampling and quantization.

II. **Why Sampling and Quantization:**

Signals available in our environment are mostly analog and continuous time. As a result we cannot use them for digital signal processing as DSP deals with discrete time and discrete valued signals. As DSP is unable to handle continuous time signals we need to take samples of the signal to make it process able with DSP. Again as the amplitude of the signal can be of any fractional value, digital computers are unable to sense all infinitesimal fractional numbers. As a result we need to quantize the discrete samples to process them with DSP.

**III. Lesson Fit:**

There is pre-requisite to this lab: CSE111, MAT120. You should have intensive Programming Knowledge and capability of understand Integral Calculus and Differential Equations.

**IV. Acceptance and Evaluation**

Performed lab tasks will be evaluated by the Lab Instructor (LI)

a. Short viva will be conducted in each Lab or occasionally to examine your work.

b. You may work in groups but be aware that you will be evaluated individually; hence active participation during the Lab work demonstration is recommended.

c. There will be Lab handout after your work you have to handover it to LI

## V. Learning Outcome:

After this Lab, the students will be able to:

a. Understand basic of continuous and discrete signals, sampling and quantization of the signal.

## VI. Activity Detail

### a. Sampling (Hour: 1)

In signal processing, sampling is the conversion of a continuous-time signal to a discrete-time signal. A common example is the conversion of a sound wave (a continuous time signal) to a sequence of samples (a discrete time signal). As computers need time to process any data, we cannot process a continuous time signal with a computer. To convert a continuous signal into a discrete signal, samples of the signal are taken periodically.

Let $x(t)$ is a continuous time signal, the corresponding discrete signal will be $x(nT)$ where $n$ = 1,2,3,4,....and $T$ is the sampling interval. We can also write the discrete signal as $x(n)$ with sampling period of $T$.

Sampling frequency is the number of samples taken per second by the continuous to discrete converter. If the sampling period is $T$, the sampling frequency will be, $f_{S=1/T}$

Let's start with a continuous-time cosine signal at 60 Hz.

```
f = 60;  % Hz
tmin = -0.05;
tmax = 0.05;
t = linspace(tmin, tmax, 400);
x_c = cos(2*pi*f * t);
plot(t,x_c)
xlabel('t (seconds)')
```

Let's sample with a sampling frequency of 800 Hz.

```
f=60;

T = 1/800;

nmin = ceil(tmin / T);

nmax = floor(tmax / T);

n = nmin:nmax;

x1 = cos(2*pi*f * n*T);

hold on
plot(n*T,x1,'.')
hold off
```

The sampling frequency of 800 Hz is well above 120 Hz, which is twice the frequency of the cosine. And you can see that the samples are clearly capturing the oscillation of the continuous-time cosine.

Let's try a lower sampling frequency.

```
T = 1/400;

nmin = ceil(tmin / T);

nmax = floor(tmax / T);

n = nmin:nmax;

x1 = cos(2*pi*f * n*T);

plot(t, x_c)

hold on
plot(n*T, x1, '.')
hold off
```

**b. Quantization (Hour 1-1:45)**

In digital signal processing, quantization is the process of approximating a continuous range of values (or a very large set of possible discrete values) by a relatively small set of discrete symbols or integer values.

Quantization is the process to convert a continuous amplitude signal into a discrete amplitude signal. To do that, finite number of levels are defined for the amplitude of the signal. As a result the discrete signal will no longer hold the values in between two levels. If any sample of the discrete signal contains any value in between two levels, the value of that point is approximated either to the nearest higher or nearest lower defined level. This process of approximation is called quantization.

As the values of the samples are approximated, the quantized signal contain more or less (depending whether the approximation is towards the higher or lower level). This is a special kind of noise. This noise is called **quantization noise.**

If a $x(n)$ discrete signal, after quantization the signal will be $y_q(n)$. Then the quantization noise will be the difference between the discrete signal and the quantized discrete signal. So, quantization noise is, $e(n) = x(n) \sim x_q(n) =$ . By theory we can define any number of quantization levels. As current day computers work with binary numbers, quantization level is defined by the power of 2 or number of bits. (i.e. a 4 bit quantization system has 16 quantum levels).

Example:
This example illustrates the nature of scalar quantization more clearly. After quantizing a sampled sine wave, it plots the original and quantized signals. The plot contrasts the x's that make up the sine curve with the dots that make up the quantized signal. The vertical coordinate of each dot is a value in the vector codebook.

```
t = [0:.1:2*pi]; % Times at which to sample the sine function
sig = sin(t); % Original signal, a sine wave
partition = [-1:.2:1]; % Length 11, to represent 12 intervals
codebook = [-1.2:.2:1]; % Length 12, one entry for each interval
[index,quants] = quantiz(sig,partition,codebook); % Quantize.
plot(t,sig,'x',t,quants,'.')
legend('Original signal','Quantized signal');
axis([-.2 7 -1.2 1.2])
```

# Lab 02

## Activity List

**Task 01:** Mark 10 **Time:** 45 minutes

**Solve the following problems with MATLAB programming language**

Question 1. How the continuous signal look like if sampling frequency is 120 Hz or 70 Hz.

Question 2. Calculate ***Root Mean Square Error*** (RMSE) from quantized signals.

---END---